

Autonomous PM Agent System: Architecture Blueprint for The Block

This report maps out a complete autonomous agent ecosystem built on the Claude Agent SDK + Obsidian vault + Zapier MCP stack, tailored to the existing infrastructure (Daily Driver agent, 106 skills, ~175 Zapier MCP tools, macOS launchd scheduling). Each proposed agent follows the same pattern already proven with the Daily Driver: skill-as-prompt, agent-as-runner, vault as shared state, launchd or event triggers, cost-capped autonomous execution.[1]

1. Sprint Health Monitor Agent

What It Does

Runs on a schedule (e.g., twice daily at 10 AM and 3 PM) and connects to Jira via the mcp-atlassian MCP server to pull sprint data, then writes a health report to the vault. The agent compares current sprint progress against expected burndown, flags tickets that haven't moved in 24+ hours, identifies tickets with no assignee or missing story points, and detects scope creep (new tickets added mid-sprint).[2][3]

Architecture

Skills: sprint-health-monitor, jira-automation, vault-read-write

Trigger: launchd (10:00 AM, 3:00 PM weekdays)

MCP Servers: vault-tools, mcp-atlassian (Jira)

Allowed Tools: Jira read operations, vault_inject, Slack send_channel_message

Output: vault/20_projects/campus-201/sprint-health/{date}.md

Alert: Slack message to #campus-dev if any ticket is blocked >24h

Key Decision Criteria for the Skill

The skill prompt should include explicit thresholds since no human is present:[1]

- **Red flag:** Any ticket in "In Progress" with no activity for 48+ hours
- **Yellow flag:** Sprint completion rate <60% with >50% of sprint elapsed
- **Scope creep alert:** >2 new tickets added after sprint start
- **Blocked indicator:** Tickets with "blocked" label or flagged status

Why This Matters for Your Week

Your 20% firefighting time largely comes from discovering problems too late. This agent acts as an early warning system — by the time you walk into standup, you already know what's stuck and can focus the conversation on unblocking rather than status updates.[4]

2. Stakeholder Report Generator Agent

What It Does

Runs weekly (Friday 3 PM, before your weekly review) and synthesizes data from Jira (sprint progress, completed tickets, velocity), Slack (key decisions, @-mentions), and Google Sheets (metrics) into a structured stakeholder update written to the vault and optionally distributed via Slack or email.[5]

Architecture

Skills: stakeholder-update, jira-automation, vault-read-write

Trigger: launchd (Friday 3:00 PM)

MCP Servers: vault-tools, mcp-atlassian, Zapier MCP (Slack, Google Sheets)

Allowed Tools: Jira read, Slack search_messages, Google Sheets read, vault_in

Output: vault/20_projects/campus-201/stakeholder-updates/{date}.md

Distribution: Slack DM to PM manager (draft for review before VP)

Data Assembly Pattern

The agent follows a gather → synthesize → write pattern:[1]

1. **Jira**: Pull completed tickets this sprint, remaining tickets, velocity vs. last sprint
2. **Slack**: Search #campus-dev and #product channels for messages containing "decision," "shipped," "blocked," or @sean mentions from the past 7 days
3. **Google Sheets**: Pull Campus metrics (users, completions, engagement) from the tracking sheet
4. **Vault**: Read this week's daily notes for context on wins, blockers, carry-forwards

Output Template

The skill should specify a precise format aligned with what your PM manager and VP expect:

- **TL;DR** (3 sentences max)
- **What shipped this week** (ticket IDs + plain-English descriptions)
- **Sprint health** (velocity, burndown status)
- **Key metrics** (with week-over-week delta)
- **Blockers & risks** (with proposed mitigations)
- **Next week focus**

This replaces the manual report assembly that currently consumes part of your Friday afternoon. Since you already have a stakeholder-update skill in the Superuser Pack, the autonomous agent version loads that same skill and adds the data-gathering layer on top.[1]

3. Meeting Intelligence Agent (Pre + Post)

The Meeting Overhead Problem

At 35% of your week in meetings (~7-8 per week, Tue-Thu concentrated), reducing meeting overhead is the highest-leverage improvement available. The Meeting Intelligence Agent operates in two modes.[6]

Pre-Meeting Mode

Trigger: launchd runs at 7:30 AM on Tue/Wed/Thu (your heavy meeting days). Agent reads Google Calendar via Zapier MCP to find today's meetings, then:

1. For each meeting, identifies attendees and pulls their recent Jira activity
2. Searches Slack for threads involving those attendees from the past 3 days
3. Reads any linked Confluence pages or Google Docs from the calendar event description
4. Writes a per-meeting brief to the vault daily note under a `<!-- meeting-prep --> anchor[1]`

Skills: meeting-prep, jira-automation, vault-read-write

Trigger: launchd (7:30 AM Tue-Thu)

MCP Servers: vault-tools, Zapier MCP (Google Calendar, Slack, Google Sheets)

Output: vault/10_timeline/daily/{date}.md under `<!-- meeting-prep --> anchor`

The brief includes: meeting purpose (from calendar description), attendee context (what they've been working on), open questions from previous meetings, relevant metrics if it's a review meeting.[7]
[8]

Post-Meeting Mode

This is harder to fully automate because transcript access depends on your meeting tool (Zoom, Google Meet). The practical pattern:

1. **If using a transcription tool** ([Otter.ai](#), Fireflies, etc.): Configure a Zapier trigger that fires when a new transcript is available, then calls a webhook that launches the agent
2. **Manual trigger alternative:** After a meeting, you paste key notes into the vault daily note under `<!-- meeting-notes -->`, then run the agent manually with `--mode post-meeting`

The post-meeting agent extracts action items, creates Jira tickets for engineering tasks, and posts a summary to the relevant Slack channel. The action item extraction follows this logic:[9][10]

- Phrases like "I'll do X," "we need to Y," "can you handle Z" become candidate action items
- Each item gets an inferred owner (from context), suggested deadline, and priority
- Engineering tasks become Jira tickets; non-engineering tasks become vault TODOs
- A confirmation message is sent to Slack: "I extracted these action items from the [meeting name] meeting — please confirm or edit"[9]

Standup Summary Distribution

Your daily standup is a prime candidate for automation. The post-standup agent can:

- Read the standup notes from the daily vault note
- Format them as a clean Slack message (per-person updates, blockers, today's focus)
- Post to #campus-dev within 5 minutes of standup ending
- This eliminates the "can you send the standup notes" follow-up loop

4. Zapier-Claude Bridge: Orchestrating Complex Workflows

The Current Limitation

Your ~175 Zapier MCP tools give Claude access to individual actions (send a Slack message, add a Google Sheets row, read a Jira ticket), but they can't trigger complex multi-step Zaps. The Zapier MCP treats each action as a standalone tool call. Meanwhile, Zapier's native logic (Paths, Filters, Delays) works well for linear workflows but struggles with conditional branching that requires reasoning.[11][12]

The Bridge Pattern

The Claude Agent SDK acts as an intelligent orchestration layer that calls Zapier MCP tools in sequences that Zapier itself can't reason about:[13][14]

Example: New Jira Ticket Enrichment Pipeline

1. Agent detects new Jira ticket created (via scheduled polling or Zapier webhook → launchd trigger)
2. Reads the ticket description, classifies it (bug, feature, task, spike)
3. If bug: searches Slack for related error reports, pulls GA4 data for affected pages, adds context to ticket
4. If feature: pulls related PRD sections from the vault, links to relevant Confluence specs, estimates scope based on similar completed tickets
5. Updates the Jira ticket with enriched context
6. Notifies the assignee via Slack with a summary

This multi-step flow requires reasoning about ticket type, context gathering from multiple sources, and conditional execution — exactly what Zapier can't do natively but what Claude excels at.[11]

Webhook-Triggered Agent Pattern

For event-driven (not just scheduled) execution, use Zapier as the event listener and Claude Agent SDK as the executor:

1. Zapier watches for an event (new Jira comment, Slack mention, Google Sheet update)
2. Zapier sends a webhook to a local endpoint (or writes a trigger file)
3. A lightweight Python watcher script detects the trigger and launches the appropriate agent
4. The agent runs with full Claude reasoning + MCP tool access[15][16]

This is more complex than launchd scheduling but enables real-time responsiveness. The `config.toml` pattern from the existing SDK setup already supports per-agent configuration; you'd add a `trigger_type = "webhook"` field.[1]

5. Product Metrics Monitor & Anomaly Detection Agent

What It Does

Runs daily at 8 AM and pulls Campus product metrics from GA4 (via Zapier MCP) and Google Sheets, writes a trend summary to the vault, and flags anomalies.[17][18]

Architecture

Skills: metrics-monitor, vault-read-write
Trigger: launchd (8:00 AM daily, expanded report Friday 8 AM)
MCP Servers: vault-tools, Zapier MCP (GA4, Google Sheets)
Output: vault/20_projects/campus-201/metrics/{date}.md
Alert: Slack DM if anomaly detected

Metrics to Track

Given your GA4/Looker access limitations (no direct API — Zapier as the bridge), focus on what's accessible:[6]

- **Daily active users** on Campus
- **Course completion rates** (overall and per-course)
- **Page views** on key content pages
- **Signup/registration funnel** conversion
- **Session duration** trends

Anomaly Detection Approach

Since the agent runs via Claude (not a dedicated ML pipeline), use a practical statistical approach rather than full ARIMA modeling:[17]

1. **Rolling average comparison:** Agent reads the last 14 daily metrics from the vault, calculates a 7-day rolling average, and flags any day where a metric deviates by more than 2 standard deviations
2. **Week-over-week comparison:** Flags if any metric drops >20% compared to the same day last week

3. **Zero/near-zero detection:** Flags if any normally non-zero metric drops to zero (indicates tracking breakage, not a real trend)

The skill prompt should include explicit thresholds and severity levels:

ANOMALY SEVERITY:

- CRITICAL: Any metric drops to 0 or near-0 (likely tracking issue) → Slack alert
- WARNING: >2 standard deviations from 14-day rolling average → Include in daily summary
- INFO: >15% week-over-week change → Include in daily summary only

Weekly Trend Summary

The Friday run generates an expanded report:

- Week-over-week trend for all tracked metrics
- Best/worst performing content or courses
- Correlation notes (e.g., "DAU spiked on days when new courses were published")
- Written to vault and sent as a formatted Slack message to your PM manager

This is a significant upgrade over manually checking GA4 dashboards or Looker (which you have limited access to anyway).[19][6]

6. PRD & Spec Automation Agent

Progressive Refinement Pattern

This agent works in two modes, following the iterative refinement approach used in multi-agent PRD systems:[20][21]

Mode 1: Rough-to-Structured (On-Demand)

You jot rough notes in the vault (under vault/20_projects/campus-201/specs/drafts/). The agent:

1. Reads the rough notes
2. Identifies what type of document it should become (PRD, tech spec, user story, spike)

3. Generates a structured first draft using the existing prd-generation skill from the Superuser Pack
4. Writes the draft to vault/20_projects/campus-201/specs/drafts/{name}-v1.md
5. Tags it with #needs-review in the frontmatter

Mode 2: Spec-Jira Sync (Scheduled)

Runs daily, checks if any Jira tickets linked to a spec have been updated (status change, new comments, scope change). If so:

1. Reads the current spec from the vault
2. Reads the Jira ticket updates
3. Appends a changelog entry to the spec: "2026-02-22: Ticket CAMPUS-142 moved to Done. Acceptance criteria 2 and 3 verified."
4. If a ticket's scope changed significantly, flags the spec as #needs-update[22]

Skills: prd-generation, jira-automation, vault-read-write

Trigger: launchd (daily 9 AM for sync mode; manual for rough-to-structured

MCP Servers: vault-tools, mcp-atlassian

Output: vault/20_projects/campus-201/specs/

Why This Matters

You already have PRD generation as an interactive skill. The autonomous layer adds two things: (1) it picks up your rough notes and turns them into drafts without you explicitly asking, and (2) it keeps specs alive by syncing with Jira rather than letting them become stale documents.[21][23]

7. Developer Coordination Agent

What It Does

Monitors GitHub for PR activity, summarizes code changes in PM-friendly language, flags blocked PRs, and generates release notes. This is the agent that bridges the gap between engineering activity and PM awareness.[24][25][26]

Architecture

Skills: dev-coordination, vault-read-write

Trigger: launchd (9 AM daily for digest, on-release for release notes)

Tools: Bash (gh CLI commands — whitelisted), vault_inject, Zapier MCP (Slack)

Output: vault/20_projects/campus-201/dev-digest/{date}.md

Alert: Slack message for blocked PRs

Daily PR Digest

The agent uses the gh CLI (already available on macOS) to pull PR data:[1]

1. gh pr list --state open --json
title,author,createdAt,reviewDecision,labels — get all open PRs
2. For each PR, check: days open, review status, CI status, requested reviewers
3. Flag: PRs open >3 days with no review, PRs with failing CI, PRs with merge conflicts
4. Summarize each PR's changes in plain English (read the PR description + diff summary)
5. Write the digest to the vault daily note

Safety note: The existing SDK architecture restricts Bash access by default. For this agent, you'd whitelist specific gh commands in the allowed_tools list rather than granting broad Bash access:[1]

```
allowed_tools=["Read", "Write", "Edit", "Glob", "Grep",  
              "mcp__vault-tools__vault_inject",  
              "Bash(gh pr list*)", "Bash(gh pr view*)", "Bash(gh release*)"]
```

Release Notes Generation

When a release is tagged (detected via scheduled polling of gh release list), the agent:

1. Pulls all merged PRs since the last release
2. Categorizes changes: features, bug fixes, infrastructure, documentation

- 3. Writes customer-facing release notes (plain language) and internal release notes (technical detail)
- 4. Posts internal notes to Slack #campus-dev
- 5. Writes both versions to vault/20_projects/campus-201/releases/[25][27]

The key insight from GitHub's own release notes API is that you can use labels to automatically categorize PRs — if your team labels PRs as feature, bugfix, infra, etc., the agent can sort them automatically. [25]

8. Implementation Roadmap

Given your current infrastructure and the phased approach already outlined in the SDK integration guide, here's a practical build order: [1]

Phase 1: Immediate (This Week)

Agent	Effort	Why First
Sprint Health Monitor	Low	Jira MCP integration is already documented in the SDK guide. Direct extension of existing jira-automation skill. Biggest impact on firefighting reduction.
Pre-Meeting Prep	Low	Uses only Zapier MCP tools you already have (Calendar, Slack). Writes to existing vault anchors. Immediate time savings on Tue-Thu.

Phase 2: Next 2 Weeks

Agent	Effort	Why Second
Stakeholder Report Generator	Medium	Combines multiple data sources. Requires tuning the synthesis prompt. Can iterate on output format.
Metrics Monitor (Basic)	Medium	Start with Google Sheets pulls only (simpler than GA4 via Zapier). Add anomaly detection after baseline is established.

Phase 3: Weeks 3-4

Agent	Effort	Why Third
Dev Coordination (PR Digest)	Medium	Requires whitelisting Bash/gh commands — slightly different security model.
Post-Meeting Action Items	Medium-High	Depends on transcript availability. Start with manual-trigger mode.
PRD Spec Sync	Medium	Requires linking vault specs to Jira tickets (metadata convention).

Phase 4: Ongoing Refinement

Agent	Effort	Why Later
Zapier Bridge (webhook-triggered)	High	Requires building the webhook listener infrastructure.
Anomaly Detection (Advanced)	High	Requires enough historical data in the vault to establish baselines.
Release Notes Generator	Medium	Depends on team's PR labeling discipline.

9. Cross-Agent Patterns and Shared Infrastructure

Vault as Shared State

Every agent reads from and writes to the same Obsidian vault, which creates a powerful compounding effect. The Sprint Health Monitor's output becomes input for the Stakeholder Report Generator. The Metrics Monitor's anomaly flags feed into the Pre-Meeting Prep agent when you have a metrics review meeting. The Dev Coordination agent's PR summaries appear in the daily planning note alongside Jira sprint data.[1]

Anchor-Based Non-Conflicting Writes

The existing `vault_inject` MCP tool uses HTML comment anchors to append content without overwriting. Each new agent gets its own anchor in the daily note template:[1]

```
<!-- sprint-health -->
<!-- meeting-prep -->
<!-- metrics-summary -->
<!-- dev-digest -->
<!-- action-items -->
```

Config.toml Convention

Each agent follows the same `config.toml` pattern, making the system predictable and maintainable:

```
[agents.sprint_health]
enabled = true
skills = ["sprint-health-monitor", "jira-automation", "vault-read-write"]
max_turns = 25
max_budget_usd = 0.40
```

Cost Management

With the existing \$0.50 default budget cap per run, running 6+ agents daily requires attention to total cost. Estimated daily costs at current Claude pricing:[1]

- Sprint Health (2x/day): ~\$0.60
- Meeting Prep (3x/week): ~\$0.30
- Metrics Monitor (1x/day): ~\$0.25
- Stakeholder Report (1x/week): ~\$0.40
- Dev Digest (1x/day): ~\$0.30
- **Estimated weekly total:** ~\$10-15

Monitor via the existing agent-run-history.csv and adjust max_budget_usd per agent based on actual usage.[1]

Safety: The Principle of Least Privilege

Each agent should only have access to the tools it needs. The Sprint Health Monitor needs Jira read access but not Jira write access. The Metrics Monitor needs Google Sheets read but never write. Define allowed_tools explicitly per agent — this is the most important safety mechanism for autonomous operation.[1]

10. Key Technical Decisions

Zapier MCP vs. Native MCP Servers

For Jira and GitHub, native MCP servers (mcp-atlassian, gh CLI) give you richer API access and don't count against Zapier task limits. Use Zapier MCP for tools where native MCP servers don't exist or aren't worth setting up (Google Calendar, Google Sheets, GA4, Slack).[11][1]

Scheduled vs. Event-Driven

Start with launchd scheduling for everything — it's proven, simple, and your infrastructure already supports it. Move to event-driven (webhook) triggers only when the latency of scheduled polling becomes a real problem. The one exception: post-meeting action item extraction genuinely benefits from event-driven triggering (when a transcript becomes available).[1]

Autonomous Decision Quality

The single most important factor for autonomous agents is the quality of decision criteria in the skill prompt. The Daily Driver agent works because its skill defines exactly what to do in each mode. Every new agent skill should include:[1]

- Explicit thresholds for flagging/alerting
- Fallback behavior when data is missing
- Output format specified precisely (with example)
- Anchor names for vault injection
- What NOT to do (prevent scope creep in agent behavior)

References

1. [agents-sdk.md](#) - # Agents SDK Integration Guide

How the Claude Agent SDK extends the Superuser Pack from an intera...

2. [Boosting Sprint Velocity with Agentic AI and JIRA Integration](#) - Agentic AI brings intelligent decision-making directly into your JIRA workflows, predicting outcomes...
3. [Jira AI Agents: Streamlining Project Management](#) - Jira AI Agents automate task management, provide predictive insights, and enhance collaboration, hel...
4. [10 AI Agents for Product Managers - MindStudio](#) - This agent helps process all the inputs and suggests optimal sprint compositions. What it does: Revi...
5. [How Agentic AI Automates Jira, Sprint Planning, and Feedback Loops](#) - Let Your Product Build Itself: How Agentic AI Automates Jira, Sprint Planning, and Feedback Loops.
6. [Sean-Winslow-Full-Personal-Context-v1.1.md](#) - # Sean Winslow — Personal Context Document
Version: 1.1 | **Last Updated:** February 15, 2026 | ...
7. [AI Agents for Meeting Preparation - ZBrain](#) - ZBrain meeting preparation agent automates the process of gathering and organizing relevant informat...
8. [Meeting Prep Agent - Lyzr AI](#) - Executives spend hours creating summaries and decks that could be automated, leaving less time

for s...

9. [Automating Action-Item Extraction and Task Assignment with AI](#) - AI can close that gap by turning spoken commitments into verified, assigned tasks that live where yo...
10. [Boost Meeting Productivity with AI-Powered Action Item Capture](#) - Our AI agents pick up meeting recordings automatically, extract the action items, parse them into ou...
11. [Zapier Workflow Skill for Claude \(Trained & Ready to Use\)](#) - Download a free, pre-built Claude Code Skill for Zapier automation. Give Claude persistent memory fo...
12. [Zapier MCP: Perform 30000+ actions in your AI tool](#) - Easily connect Zapier MCP to ChatGPT or Claude in minutes without coding or technical setup and then...
13. [Zapier Workflows Claude Code Skill | AI Automation - MCP Market](#) - The Zapier Workflows skill bridges the gap between Claude Code and Zapier's ecosystem of 8,000+ tool...
14. [Zapier Workflows & MCP Orchestration | Claude Code Skill](#) - This skill transforms Claude into a high-functioning automation agent by bridging the gap between on...
15. [Event-Driven Agentic Loop with Claude Code with Ryan Sweet](#) - ... agent workflows across different layers of complexity. Whether you're using Claude or other fram...
16. [How Event-Driven Workflows Work](#) - Event-driven workflows are a system that responds instantly to specific triggers, bypassing traditio...
17. [Automated Anomaly Detection in GA4 Data Streams - Refresh Agent](#) - Detect GA4 anomalies with ARIMA and Isolation Forest via the realtime API. Protect revenue from tagg...
18. [Google Analytics 4 AI Features For Smarter Marketing Decisions](#) - Google Analytics 4 addresses this gap through AI-powered anomaly detection, designed to alert market...
19. [How to Build an AI Dashboard in GA4 for Smarter Marketing Decisions](#) - Discover how an AI dashboard in GA4 turns analytics into clear, actionable marketing insights. Build...
20. [\(NEW\) \[Product Initiative\] Requirements Gathering Automation with ...](#) - Requirement Refinement: Agents iteratively refine requirements based on feedback and collaboration, ...
21. [Multi-agent PRD automation with MetaGPT, Ollama, and DeepSeek](#) - By following this tutorial, you've learned how to

- automate the creation and refinement of a product ...
22. [Architectural Blueprint for the Closed-Loop Autonomous Agent ...](#) - Step 1: Deep Research and Structured Product Requirements Document (PRD) Generation. The initial pha...
 23. [Building an internal agent: Iterative prompt and skill refinement](#) - Some of our internal workflows are being used quite frequently, and usage reveals gaps in the curren...
 24. [AI Release Notes · GitHub Marketplace](#) - This app automatically generates comprehensive and coherent release notes for each new release in yo...
 25. [DevOps: Enhanced Release Automation with GitHub's AI-Powered ...](#) - GitHub provides a powerful API that generates release notes automatically based on pull requests, la...
 26. [GitHub Release Notes Agent - Agno](#) - AI Agent. GitHub Release Notes Agent. Automatically turn commits, pull requests, and diffs into poli...
 27. [How to Automate Release Notes with AI - Ascend.io](#) - Build an automated release notes pipeline using GitHub Actions and OpenAI. Transform manual document...